

■ 2.6.20 Special Topic: Reading Data from Files and Pipes

With `<<` you can read the contents of a file as *Mathematica* input. However, if you have data that has been generated by an external program, it may not be in exactly the right form to be used as standard *Mathematica* input. For example, it may consist of a sequence of numbers, one to a line. You can use `ReadList` to read in such a sequence of numbers, and convert it to a *Mathematica* list.

<code>ReadList["file", type]</code>	read a sequence of objects of a particular type from a file, and put them in a <i>Mathematica</i> list
<code>ReadList["!command", type]</code>	execute an external program, and read the objects it produces

Reading lists of data.

<code>Byte</code>	single byte of data, returned as an integer
<code>Character</code>	single character, returned as a one-character string
<code>String</code>	character string, terminated by a newline (line feed or return)
<code>Real</code>	approximate number in FORTRAN-like "E" notation
<code>Number</code>	exact or approximation number in FORTRAN-like notation
<code>Expression</code>	<i>Mathematica</i> expression
<code>{type₁, type₂, ...}</code>	a sequence of objects of various types

Types of objects to read.

The file `cubes.dat` contains a table of numbers. The numbers in scientific notation are given in the form they would be generated by C or FORTRAN.

```
In[1]:= !!cubes.dat
128      2.09715E+6
256      1.67772E+7
512      1.34218E+8
```

This reads in the numbers in `cubes.dat`, and puts them all in a list.

```
In[1]:= ReadList["cubes.dat", Number]

Out[1]= {128, 2.09715 106, 256, 1.67772 107, 512,
1.34218 108}
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This reads in each line of `cubes.dat` as a string. Using `InputForm` makes the form of the strings clear.

```
In[2]:= ReadList["cubes.dat", String] // InputForm
Out[2]//InputForm=
{"128      2.09715E+6", "256      1.67772E+7",
 "512      1.34218E+8"}
```

This reads the numbers in `cubes.dat` in pairs, giving as a result a list of the pairs.

```
In[3]:= ReadList["cubes.dat", {Number, Number}]
Out[3]= {{128, 2.09715 106}, {256, 1.67772 107},
         {512, 1.34218 108}}
```

You can specify any list structure in which to insert the numbers that you read.

```
In[4]:= ReadList["cubes.dat", {{Number}, {Number}}]
Out[4]= {{{128}, {2.09715 106}}, {{256}, {1.67772 107}},
         {{512}, {1.34218 108}}}
```

If you use the type `Number`, *Mathematica* will give any number that does not contain an explicit decimal point as an integer. If you use `Real` instead, `ReadList` will give all the numbers it reads in approximate form. Notice that just as `ReadList` converts numbers from C or FORTRAN form to *Mathematica* form, so you can also use `CForm` and `FortranForm` to convert numbers in *Mathematica* to C and FORTRAN form.

<code>ReadList["file", Number]</code>	read a sequence of numbers from a file
<code>ReadList["file", {Number, Number}]</code>	read pairs of numbers from a file
<code>ReadList["file", Table[Number, {n}]]</code>	read numbers in blocks of <i>n</i> from a file

Typical ways to read numerical data.

The function `ReadList` works not only for reading data from a file, but also for reading data generated by an external program.

The external program `squares` *n* prints out numbers and their squares.

```
In[5]:= !squares 3
1 1
2 4
3 9
```

This runs the external program `squares` 4, then reads in pairs of numbers from the output, and puts the result in a list.

```
In[5]:= ReadList["!squares 4", {Number, Number}]
Out[5]= {{1, 1}, {2, 4}, {3, 9}, {4, 16}}
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

`ReadList` allows you to read in *all* the data from a file or pipe. Sometimes you may want to read data from a file one piece at time, perhaps doing tests each time to tell what kind of data to expect next.

<code>OpenRead["file"]</code>	open a file for reading
<code>Read["file"]</code>	read an expression from a file
<code>Read["file", type]</code>	read an object of the specified type from a file
<code>Close["file"]</code>	close a file

Functions for reading objects from files.

When you read individual pieces of data from files, *Mathematica* always remembers the “current point” in the file that you are at. When you call `OpenRead`, the current point is at the beginning of the file. Every time you call `Read` to read an object from the file, the current point moves on. If you try to read past the end of the file, `Read` returns the symbol `EndOfFile`. You can get back to the beginning of the file by calling `Close`, and then calling `OpenRead` again.

This opens the file `cubes.dat` for reading.

```
In[6]:= f = OpenRead["cubes.dat"]
Out[6]= cubes.dat
```

This reads one number from `cubes.dat`.

```
In[7]:= Read[f, Number]
Out[7]= 128
```

This reads a string from the file. `InputForm` makes the form of the string clear.

```
In[8]:= Read[f, String] // InputForm
Out[8]//InputForm= "      2.09715E+6"
```

This reads in the remaining numbers in the file, and puts them in a list.

```
In[9]:= ReadList[f, Number]
Out[9]= {256, 1.67772 107, 512, 1.34218 108}
```

This closes the file.

```
In[10]:= Close[f]
Out[10]= cubes.dat
```

If you call `Read` a few times, and then call `ReadList`, *Mathematica* will read from where you are to the end of the file, giving a list of the objects read.

Like `ReadList`, `Read` can also be used to read through pipes objects generated as output from external programs.